System Verification and Validation Plan for Lake

Team #10, CinqGarçons
Ethan Walsh
Colin Chambachan
Matthew Nesbitt
Jinwoo Hong
Gary Qin

October 28, 2026

Revision History

Date	Version	Notes
Date 1	1.0	Notes
Date 2	1.1	Notes

[The intention of the VnV plan is to increase confidence in the software. However, this does not mean listing every verification and validation technique that has ever been devised. The VnV plan should also be a **feasible** plan. Execution of the plan should be possible with the time and team available. If the full plan cannot be completed during the time available, it can either be modified to "fake it", or a better solution is to add a section describing what work has been completed and what work is still planned for the future. —SS]

[The VnV plan is typically started after the requirements stage, but before the design stage. This means that the sections related to unit testing cannot initially be completed. The sections will be filled in after the design stage is complete. the final version of the VnV plan should have all sections filled in. —SS]

Contents

1 Symbols, Abbreviations, and Acronyms **iv**

2 General Information **1**
- 2.1 Summary . . . 1
- 2.2 Objectives . . . 1
- 2.3 Custom Documents . . . 1
- 2.4 Relevant Documentation . . . 2

3 Plan **2**
- 3.1 Verification and Validation Team . . . 2
- 3.2 SRS Verification . . . 2
- 3.3 Design Verification . . . 2
- 3.4 Verification and Validation Plan Verification . . . 3
- 3.5 Implementation Verification . . . 3
- 3.6 Automated Testing and Verification Tools . . . 3
- 3.7 Software Validation . . . 4

4 System Tests **4**
- 4.1 Tests for Functional Requirements . . . 4
  - 4.1.1 Area of Testing1 . . . 4
  - 4.1.2 Area of Testing2 . . . 5
- 4.2 Tests for Nonfunctional Requirements . . . 5
  - 4.2.1 Area of Testing1 . . . 6
  - 4.2.2 Area of Testing2 . . . 6
- 4.3 Traceability Between Test Cases and Requirements . . . 7

5 Unit Test Description **7**
- 5.1 Unit Testing Scope . . . 7
- 5.2 Tests for Functional Requirements . . . 7
  - 5.2.1 Module 1 . . . 7
  - 5.2.2 Module 2 . . . 8
- 5.3 Tests for Nonfunctional Requirements . . . 9
  - 5.3.1 Module ? . . . 9
  - 5.3.2 Module ? . . . 9
- 5.4 Traceability Between Test Cases and Modules . . . 9

6 Appendix **11**
6.1 Symbolic Parameters 11
6.2 Usability Survey Questions? 11

7 Appendix — Generative AI Use Disclosure **12**
7.1 AI Tools Used 12
7.2 How and Where Used 12
7.3 Verification of AI-Generated Content 12

8 Appendix — Reflection **13**

List of Tables

[Remove this section if it isn't needed —SS]

List of Figures

[Remove this section if it isn't needed —SS]

1 Symbols, Abbreviations, and Acronyms

Term	Description
Activation	An intermediate value that a model layer computes and passes to the next layer.
AI	Artificial intelligence. Software that does tasks that normally need human judgement. In this document it always means an LLM.
API	Application programming interface. A set of requests one program can make of another. Lake's API is the OpenAI-compatible API on the leader node.
ARM	One of the two CPU architectures Lake supports, used by Apple silicon and most mobile devices.
Backpressure	A signal that a node is receiving tensors faster than it can process them, so the upstream node should slow down.
Bottleneck	The node or network connection that takes the longest per token and limits the throughput of the whole Lake network.
Context window	The maximum number of tokens, prompt and response together, that a model can handle in one request.
Continuous integration	Building and testing the code automatically on every push to the repository.
Coordinator	The Lake component that forms and monitors the Lake network, serves the API on the leader node, and manages the Inference Engine on each node.
CPU	Central processing unit. A device's general-purpose processor.
Device owner	The person whose computer joins a Lake network. They must consent before their device takes part.

Term	Description
Dynamic allocation	Moving model layers between nodes while the Lake network is running, when a node joins, leaves, or fails.
FFI	Foreign function interface. An interface that lets code in one language call code in another.
FLOP/s	Floating-point operations per second. The number of floating-point operations a processor performs per second, used to measure compute performance.
Follower node	Any node in a Lake network other than the leader node.
GGUF	The file format used by llama.cpp and ggml to store a model's weights, tokeniser, and metadata. Lake only accepts GGUF models.
GPL	GNU General Public License. The free software licence, version 3.0, that Lake is released under.
GPU	Graphics processing unit. A device's parallel processor, which does the arithmetic of inference.
Greedy decoding	Always choosing the most probable next token, so the same prompt gives the same output every time.
gRPC	A remote procedure call framework from Google. Tensor I/O uses it to send tensors between nodes.
HTTP	Hypertext Transfer Protocol. The request and response protocol of the web.
Inference	Using a model to generate text from a prompt.
Inference Engine	The Lake component on each node that loads the node's partition and runs it to generate output. It is built on ggml.
Inference user	A person, or a tool acting for one, who sends prompts to a Lake network and gets responses back.
IPC	Inter-process communication. Passing data between processes on the same machine.

Term	Description
KV cache	The key and value tensors each layer stores for past tokens so they are not recomputed. Each node keeps the KV cache for its own layers.
Lake	The system this document specifies. It runs one LLM across several devices on a local network using model layer partitioning.
Lake Client	The Lake application installed on each device, and the component people use. Every node runs one. The operator forms a Lake network, loads a model, and views performance from the Lake Client on the leader node, and a device owner confirms joining from the Lake Client on their own device.
Lake network	The nodes on one local network that have joined to serve one model.
Latency	The time between the start of an inference request and a defined event, such as the first token or the end of the response.
Layer	One of the stacked stages of a transformer model. Lake splits a model between nodes at layer boundaries.
Leader node	The node whose operator formed the Lake network. It runs the Model Partitioner, serves the API, and monitors the followers.
LLM	Large language model. An AI model trained on a large body of text and used to generate text.
Model layer partitioning	Splitting an LLM's layers into consecutive ranges and giving each range to a different node.
Model Partitioner	The Lake component on the leader node that decides which layers each node holds, based on each node's memory, compute, and network performance.

Term	Description
MoSCoW	A prioritisation scheme that sorts requirements into Must have, Should have, Could have, and Won't have.
NIST	National Institute of Standards and Technology. The United States standards agency.
Node	A device running the Lake Client that has joined a Lake network.
OCR	Optical character recognition. Converting images of text into machine-readable text.
OpenAI-compatible API	An HTTP API with the same request and response formats as the OpenAI API, so clients written for OpenAI work with Lake unchanged.
Operator	The person who forms a Lake network, loads a model, and monitors performance from the Lake Client. Also called the Lake operator.
P2P	Peer-to-peer. A direct connection between two nodes that does not go through the leader. Neighbouring nodes send tensors this way.
Partition	A consecutive range of a model's layers assigned to one node.
Partitioning scheme	The assignment of partitions to nodes for a whole model.
PEGS	Meyer's four-chapter structure for a requirements document: Project, Environment, Goals, and System.
Perplexity	A measure of how well a model predicts a body of text, where lower is better. Used to compare a split model with the unsplit one.
PIPEDA	Personal Information Protection and Electronic Documents Act. Canada's private-sector privacy law.
PoC	Proof of concept. The first working demonstration of the system.

Term	Description
Prefill	The first phase of a request, where the whole prompt runs through the model to fill the KV cache before the first output token.
Quantisation	Converting a model's weights to a lower-precision format to reduce memory use.
RACI	A chart showing who is Responsible, Accountable, Consulted, and Informed for each piece of work.
REST	Representational state transfer. A style of HTTP API where clients read and change named resources.
RPC	Remote procedure call. Calling a function on another machine as if it were local.
RTT	Round-trip time. The time for a message to reach another node and for the reply to come back.
Session	A period of interaction with Lake in which one or more inference requests are served. For the operator, it runs from forming a Lake network to shutting it down. For an inference user, it is one conversation whose context is kept between prompts.
SRS	Software Requirements Specification. This document.
Streaming	Returning a response token by token as it is generated.
TA	Teaching assistant. The course staff member who advises the team and assesses its deliverables.
Telemetry	Usage statistics, crash reports, update checks, or any other data that software sends to its developer or a third party. Lake sends none.
Tensor	A multi-dimensional array of numbers. Weights, activations, and the KV cache are all tensors.
Tensor I/O	The Lake component that compresses, sends, receives, and decompresses tensors between neighbouring nodes.

Term	Description
Token	A unit of text processed by an LLM, typically a word or part of a word.
Token throughput	The number of tokens generated per unit time, usually tokens per second.
Tool calling	An OpenAI API feature where the model asks the client to run a named function and return the result.
Transformer	The neural network architecture of current LLMs, built from a stack of layers.
UI	User interface. The part of a program a person interacts with. Lake's UI is the Lake Client.
V&V	Verification and validation. Verification checks that the system meets its requirements. Validation checks that it is the right system for its stakeholders.
Weights	The learned parameters of a model that determine its behaviour. Lake never modifies them.
x86_64	The 64-bit instruction set of Intel and AMD processors. One of the two CPU architectures Lake supports.

This document ... [provide an introductory blurb and roadmap of the Verification and Validation plan —SS]

2 General Information

2.1 Summary

[Say what software is being tested. Give its name and a brief overview of its general functions. —SS]

2.2 Objectives

[State what is intended to be accomplished. The objective will be around the qualities that are most important for your project. You might have something like: "build confidence in the software correctness," "demonstrate adequate usability." etc. You won't list all of the qualities, just those that are most important. —SS]

[You should also list the objectives that are out of scope. You don't have the resources to do everything, so what will you be leaving out. For instance, if you are not going to verify the quality of usability, state this. It is also worthwhile to justify why the objectives are left out. —SS]

[The objectives are important because they highlight that you are aware of limitations in your resources for verification and validation. You can't do everything, so what are you going to prioritize? As an example, if your system depends on an external library, you can explicitly state that you will assume that external library has already been verified by its implementation team. —SS]

2.3 Custom Documents

[Summarize the CDs (Custom Documents) that were tackled by this project. CDs can include usability testing, code walkthroughs, user documentation, formal proof, GenderMag personas, Design Thinking, etc. CDs should have already been approved by the course instructor as included in your problem statement. You can use a pull request to update your CDs (in TeamComposition.csv or Repos.csv) if your plan changes as a result of the VnV planning exercise. —SS]

2.4 Relevant Documentation

[Reference relevant documentation. This will definitely include your SRS and your other project documents (design documents, like MG, MIS, etc). You can include these even before they are written, since by the time the project is done, they will be written. You can create BibTeX entries for your documents and within those entries include a hyperlink to the documents. —SS]

Author (2019)

[Don't just list the other documents. You should explain why they are relevant and how they relate to your VnV efforts. —SS]

3 Plan

[Introduce this section. You can provide a roadmap of the sections to come. —SS]

3.1 Verification and Validation Team

[Your teammates. Maybe your supervisor. You should do more than list names. You should say what each person's role is for the project's verification. A table is a good way to summarize this information. —SS]

3.2 SRS Verification

[List any approaches you intend to use for SRS verification. This may include ad hoc feedback from reviewers, like your classmates (like your primary reviewer), or you may plan for something more rigorous/systematic. —SS]

[If you have a supervisor for the project, you shouldn't just say they will read over the SRS. You should explain your structured approach to the review. Will you have a meeting? What will you present? What questions will you ask? Will you give them instructions for a task-based inspection? Will you use your issue tracker? —SS]

[Maybe create an SRS checklist? —SS]

3.3 Design Verification

[Plans for design verification —SS]

[The review will include reviews by your classmates —SS]
[Create a checklists? —SS]

3.4 Verification and Validation Plan Verification

[The verification and validation plan is an artifact that should also be verified. Techniques for this include review and mutation testing. —SS]
[The review will include reviews by your classmates —SS]
[Create a checklists? —SS]

3.5 Implementation Verification

[You should at least point to the tests listed in this document and the unit testing plan. —SS]

[In this section you would also give any details of any plans for static verification of the implementation. Potential techniques include code walkthroughs, code inspection, static analyzers, etc. —SS]

[The final class presentation in CAS 741 could be used as a code walkthrough. There is also a possibility of using the final presentation (in CAS741) for a partial usability survey. —SS]

3.6 Automated Testing and Verification Tools

[What tools are you using for automated testing. Likely a unit testing framework and maybe a profiling tool, like ValGrind. Other possible tools include a static analyzer, make, continuous integration tools, test coverage tools, etc. Explain your plans for summarizing code coverage metrics. Linters are another important class of tools. For the programming language you select, you should look at the available linters. There may also be tools that verify that coding standards have been respected, like flake9 for Python. —SS]

[If you have already done this in the development plan, you can point to that document. —SS]

[The details of this section will likely evolve as you get closer to the implementation. —SS]

3.7 Software Validation

[If there is any external data that can be used for validation, you should point to it here. If there are no plans for validation, you should state that here. —SS]

[You might want to use review sessions with the stakeholder to check that the requirements document captures the right requirements. Maybe task based inspection? —SS]

[For those capstone teams with an external supervisor, the Rev 0 demo should be used as an opportunity to validate the requirements. You should plan on demonstrating your project to your supervisor shortly after the scheduled Rev 0 demo. The feedback from your supervisor will be very useful for improving your project. —SS]

[For teams without an external supervisor, user testing can serve the same purpose as a Rev 0 demo for the supervisor. —SS]

[This section might reference back to the SRS verification section. —SS]

4 System Tests

[There should be text between all headings, even if it is just a roadmap of the contents of the subsections. —SS]

4.1 Tests for Functional Requirements

[Subsets of the tests may be in related, so this section is divided into different areas. If there are no identifiable subsets for the tests, this level of document structure can be removed. —SS]

[Include a blurb here to explain why the subsections below cover the requirements. References to the SRS would be good here. —SS]

4.1.1 Area of Testing1

[It would be nice to have a blurb here to explain why the subsections below cover the requirements. References to the SRS would be good here. If a section covers tests for input constraints, you should reference the data constraints table in the SRS. —SS]

Title for Test

1. test-id1

   Control: Manual versus Automatic

   Initial State:

   Input:

   Output: [The expected result for the given inputs. Output is not how you are going to return the results of the test. The output is the expected result. —SS]

   Test Case Derivation: [Justify the expected value given in the Output field —SS]

   How test will be performed:

2. test-id2

   Control: Manual versus Automatic

   Initial State:

   Input:

   Output: [The expected result for the given inputs —SS]

   Test Case Derivation: [Justify the expected value given in the Output field —SS]

   How test will be performed:

4.1.2 Area of Testing2

...

4.2 Tests for Nonfunctional Requirements

[The nonfunctional requirements for accuracy will likely just reference the appropriate functional tests from above. The test cases should mention reporting the relative error for these tests. Not all projects will necessarily have nonfunctional requirements related to accuracy. —SS]

[For some nonfunctional tests, you won't be setting a target threshold for passing the test, but rather describing the experiment you will do to measure the quality for different inputs. For instance, you could measure speed versus the problem size. The output of the test isn't pass/fail, but rather a summary table or graph. —SS]

[Tests related to usability could include conducting a usability test and survey. The survey will be in the Appendix. —SS]

[Static tests, review, inspections, and walkthroughs, will not follow the format for the tests given below. —SS]

[If you introduce static tests in your plan, you need to provide details. How will they be done? In cases like code (or document) walkthroughs, who will be involved? Be specific. —SS]

4.2.1 Area of Testing1

Title for Test

1. test-id1

   Type: Functional, Dynamic, Manual, Static etc.

   Initial State:

   Input/Condition:

   Output/Result:

   How test will be performed:

2. test-id2

   Type: Functional, Dynamic, Manual, Static etc.

   Initial State:

   Input:

   Output:

   How test will be performed:

4.2.2 Area of Testing2

...

4.3 Traceability Between Test Cases and Requirements

[Provide a table that shows which test cases are supporting which requirements. —SS]

5 Unit Test Description

[This section should not be filled in until after the MIS (detailed design document) has been completed. —SS]

[Reference your MIS (detailed design document) and explain your overall philosophy for test case selection. —SS]

[To save space and time, it may be an option to provide less detail in this section. For the unit tests you can potentially layout your testing strategy here. That is, you can explain how tests will be selected for each module. For instance, your test building approach could be test cases for each access program, including one test for normal behaviour and as many tests as needed for edge cases. Rather than create the details of the input and output here, you could point to the unit testing code. For this to work, you code needs to be well-documented, with meaningful names for all of the tests. —SS]

5.1 Unit Testing Scope

[What modules are outside of the scope. If there are modules that are developed by someone else, then you would say here if you aren't planning on verifying them. There may also be modules that are part of your software, but have a lower priority for verification than others. If this is the case, explain your rationale for the ranking of module importance. —SS]

5.2 Tests for Functional Requirements

[Most of the verification will be through automated unit testing. If appropriate specific modules can be verified by a non-testing based technique. That can also be documented in this section. —SS]

5.2.1 Module 1

[Include a blurb here to explain why the subsections below cover the module. References to the MIS would be good. You will want tests from a black box

perspective and from a white box perspective. Explain to the reader how the tests were selected. —SS]

1. test-id1

   Type: [Functional, Dynamic, Manual, Automatic, Static etc. Most will be automatic —SS]

   Initial State:

   Input:

   Output: [The expected result for the given inputs —SS]

   Test Case Derivation: [Justify the expected value given in the Output field —SS]

   How test will be performed:

2. test-id2

   Type: [Functional, Dynamic, Manual, Automatic, Static etc. Most will be automatic —SS]

   Initial State:

   Input:

   Output: [The expected result for the given inputs —SS]

   Test Case Derivation: [Justify the expected value given in the Output field —SS]

   How test will be performed:

3. ...

5.2.2 Module 2

...

5.3 Tests for Nonfunctional Requirements

[If there is a module that needs to be independently assessed for performance, those test cases can go here. In some projects, planning for nonfunctional tests of units will not be that relevant. —SS]

[These tests may involve collecting performance data from previously mentioned functional tests. —SS]

5.3.1 Module ?

1. test-id1

   Type: [Functional, Dynamic, Manual, Automatic, Static etc. Most will be automatic —SS]

   Initial State:

   Input/Condition:

   Output/Result:

   How test will be performed:

2. test-id2

   Type: Functional, Dynamic, Manual, Static etc.

   Initial State:

   Input:

   Output:

   How test will be performed:

5.3.2 Module ?

...

5.4 Traceability Between Test Cases and Modules

[Provide evidence that all modules have been considered. —SS]

References

Author Author. System requirements specification. https://github.com/..., 2019.

6 Appendix

This is where you can place additional information.

6.1 Symbolic Parameters

The definition of the test cases will call for SYMBOLIC_CONSTANTS. Their values are defined in this section for easy maintenance.

6.2 Usability Survey Questions?

[This is a section that would be appropriate for some projects. —SS]

7 Appendix — Generative AI Use Disclosure

[ChatGPT (version 5) was used to brainstorm ideas for the template for this disclose section. —SS]

7.1 AI Tools Used

[Give the name of the AI tool(s) used and the version. —SS]

7.2 How and Where Used

[Explain what the tool(s) were used for. Examples include: brainstorming, explaining a technical concept, reviewing the document, generating or modifying text, generating or modifying diagrams, identifying errors or omissions or inconsistencies. —SS]

[For each of the uses, identify which parts of the document were affected. —SS]

[You do not need to report every interaction, but you should disclose the uses that materially contributed to your work —SS]

7.3 Verification of AI-Generated Content

[

- Describe how the team checked AI-generated or AI-assisted material before including it in the document.
- In particular, verify factual claims, technical assertions, requirements, calculations, references, and other information for which an AI system may produce incorrect or fabricated results.
- **The team is responsible for the accuracy, completeness, consistency, and appropriateness of the submitted document, regardless of whether material was generated by a human or an AI system.**

—SS]

8 Appendix — Reflection

[This section is not required for CAS 741 —SS]

The information in this section will be used to evaluate the team members on the graduate attribute of Lifelong Learning.

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. What went well while writing this deliverable?

2. What pain points did you experience during this deliverable, and how did you resolve them?

3. What knowledge and skills will the team collectively need to acquire to successfully complete the verification and validation of your project? Examples of possible knowledge and skills include dynamic testing knowledge, static testing knowledge, specific tool usage, Valgrind etc. You should look to identify at least one item for each team member.

4. For each of the knowledge areas and skills identified in the previous question, what are at least two approaches to acquiring the knowledge or mastering the skill? Of the identified approaches, which will each team member pursue, and why did they make this choice?